PIM-Attn: Efficient Asymmetric Quantized Attention with Near-Bank Processing-in-Memory

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous Country
anon@example.com

***Abstract*—Large Language Models (LLMs) face a critical memory bottleneck from their key-value (KV) cache during inference. While recent works like KiVI propose tuning-free asymmetric 2-bit quantization to dramatically reduce KV cache size, the dequantization overhead on GPU introduces significant latency. Existing near-bank processing-in-memory (PIM) approaches are insufficient for the mixed-precision operations required by asymmetric quantization. We propose PIM-Attn, a heterogeneous attention framework that decomposes the asymmetric dot product into parallel 2-bit operations mapped to near-bank PIM units, while routing float16-intensive score-value operations to GPU. Cycle-accurate simulation on Llama-3-8B demonstrates $1.4\times$–$2.9\times$ per-sequence Q@K latency improvement over GPU-only execution, with $2.1\mu s$ per-sequence latency on 2-bit KV using 2048 PIM banks. Our results show that direct 2-bit PIM compute is $2\times$ more efficient than dequantize-then-compute approaches, validating the decomposition strategy.**

***Index Terms*—LLM inference, KV cache quantization, near-bank PIM, attention, heterogeneous computing**

I. Introduction

Large Language Models (LLMs) have become the backbone of modern AI applications [1]. However, their deployment at scale remains expensive due to massive computational and memory requirements. A key bottleneck in LLM inference is the key-value (KV) cache, which stores attention keys and values for all previously generated tokens to avoid recomputation [2]. In long-context and large-batch scenarios, the KV cache can exceed model parameters by multiple times: for 540B PaLM with batch size 512 and context length 2048, KV cache alone consumes 3TB of memory [2].

To address this, quantization techniques reduce the bit-width of KV cache entries. KiVI [3] proposes tuning-free asymmetric 2-bit quantization, achieving $8\times$ memory savings with per-channel quantization for keys and per-token quantization for values. However, while KiVI effectively reduces KV cache *size*, the dequantization overhead during attention computation introduces latency that partially offsets memory bandwidth savings.

The root cause is the mathematical expansion of asymmetric quantized dot products:

$$Q \cdot K_i \approx s_Q s_K \left[\sum_{j=1}^{d} \hat{q}_j \hat{k}_{ij} - z_K \sum_{j=1}^{d} \hat{q}_j - z_Q \sum_{j=1}^{d} \hat{k}_{ij} + d z_Q z_K \right] \tag{1}$$

This reveals four distinct operations beyond the standard dot product $\sum \hat{q}\hat{k}$: two channel-wise sums ($\sum \hat{q}$, $\sum \hat{k}$) and a constant correction. On GPU, these extra operations add significant overhead.

Processing-in-memory (PIM) offers a promising alternative by placing compute units close to memory [4], [5]. Prior PIM designs for attention [6] used logic-die PIM, which places compute on a separate die connected via TSVs, introducing bandwidth bottlenecks. Near-bank PIM, where lightweight PEs reside inside each DRAM bank, provides higher local bandwidth and lower latency for bank-local operations.

In this paper, we propose PIM-Attn, a heterogeneous attention framework that decomposes the asymmetric quantized attention across near-bank PIM and GPU. Our key insight: the operations in Eq. 1 have fundamentally different computational characteristics. The main dot product $\sum \hat{q}\hat{k}$ and reduction sums operate on 2-bit data — ideal for near-bank PIM with efficient 2-bit PEs. The score-value ($S \cdot V$) operation involves float16 tensors — better suited for GPU's massive float16 throughput. Softmax is lightweight and stays on GPU.

PIM-Attn maps the decomposed Q@K operations to near-bank PIM with 2-bit compute units and routes score@V to GPU. We implement a cycle-accurate simulator with per-stage timing breakdown and configurable PE microarchitecture. Evaluation on Llama-3-8B with 16K context demonstrates $2.1\mu s$ per-sequence Q@K latency on 2-bit KV with 2048 PIM banks — $1.4\times$ faster than GPU. We further show that PIM-side dequantization incurs $2\times$ overhead, validating direct 2-bit compute.

Our contributions:

- We identify the computational mismatch between asymmetric quantized attention and existing hardware, motivating a heterogeneous decomposition.
- We design PIM-Attn, mapping decomposed Q@K to near-bank PIM with 2-bit PEs, while routing float16-heavy operations to GPU.
- We build a cycle-accurate PIM simulator with configurable architecture and per-stage timing verification (ratio $1.00\times$).
- We provide comprehensive experimental analysis across six configurations.

II. Background and Motivation

A. Asymmetric Quantization for KV Cache

KiVI [3] adopts asymmetric (affine) quantization where each value maps as $x = s(\hat{x} - z)$ with per-channel scale s and

zero-point z. For 2-bit quantization, $\hat{x} \in \{0, 1, 2, 3\}$ requiring only 0.25 bytes per element — an $8\times$ reduction from float16.

The attention score between query Q and key K_i with asymmetric quantization expands as:

$$Q \cdot K_i = s_Q s_K \left[\sum_{j=1}^{d} \hat{q}_j \hat{k}_{ij} - z_K \sum_{j=1}^{d} \hat{q}_j - z_Q \sum_{j=1}^{d} \hat{k}_{ij} + d z_Q z_K \right] \tag{2}$$

This reveals four operations: (1) main dot product $\sum \hat{q}\hat{k}$ on 2-bit data, (2) Q-reduction $\sum \hat{q}$, (3) K-reduction $\sum \hat{k}$, (4) constant $dz_Q z_K$.

Key observation: operations (1)–(3) operate on quantized 2-bit data, processable by simple near-memory compute units. Only the final scaling $s_Q s_K$ requires float16.

B. Near-Bank PIM Architecture

Near-bank PIM places lightweight PEs inside each DRAM bank [5]. Each bank has: a row buffer (R bytes) serving as PE working memory, a MAC unit processing W bytes per cycle at frequency f, and local DRAM-to-rowbuffer bandwidth B GB/s.

For HBM3E with 8 stacks $\times$ 256 PIM banks, $N = 2048$ banks operate in parallel. Data is striped across banks; each bank processes its partition independently.

C. Limitations of Existing Approaches

GPU-only: KiVI's dequantization and reduction overhead on GPU limits throughput as extra terms in Eq. 2 compete for compute units.

Logic-die PIM [6]: Compute on separate logic die requires data traversal via TSVs, introducing bandwidth bottlenecks for fine-grained bank-local operations.

PIM dequantization: Dequantizing 2-bit to float16 inside PIM ($s \cdot (\hat{x} - z)$) adds $3\times$ PE time per element, doubling Q@K latency. This motivates our approach of *decomposing and keeping operations in the 2-bit domain*.

III. PIM-Attn Design

A. System Overview

PIM-Attn adopts a three-phase heterogeneous execution model. Phase 1 (PIM): decomposed Q@K dot product on near-bank PIM with 2-bit KV. Phase 2 (GPU): partial result reduction and softmax. Phase 3 (GPU): score@V matrix multiply with dequantized float16 values.

B. Decomposed Q@K on Near-Bank PIM

For a query $Q \in \mathbb{R}^{M \times d}$ and key cache $K \in \mathbb{R}^{N \times d}$ striped across N_{banks}, each bank computes its partition of Eq. 2:

1) $\sum \hat{q}_j \hat{k}_{ij}$ — 2-bit MAC, the main dot product.
2) $\sum \hat{q}_j$ — 2-bit accumulate, computed once per query.
3) $\sum \hat{k}_{ij}$ — 2-bit accumulate, per key vector.
4) Aggregate via Eq. 2 after cross-bank reduction.

Operations (1)–(3) all use 2-bit data, making them efficient on simple PIM PEs. The final float16 scaling $s_Q s_K$ is applied after reduction.

C. PIM Latency Model

We model per-bank latency using a pipelined row-buffer execution model. Data is loaded from DRAM into the row buffer (R bytes), then the PE computes on it. While PE computes on buffer i, buffer $i+1$ fills from DRAM.

Total data per GEMV: $B_{\text{total}} = (MK + KN + MN) \cdot e$ bytes, where e is element size. Per bank: $B_{\text{bank}} = B_{\text{total}} / N_{\text{banks}}$. Row buffer fills: $n_{\text{rb}} = \lceil B_{\text{bank}} / R \rceil$, each taking $t_{\text{fill}} = R/B$ ns. PE time: $T_{\text{pe}} = B_{\text{bank}} / W \cdot \tau$, where τ is the PE cycle time.

When asymmetric quantization is enabled, reduction terms add PE-only overhead:

$$T_{\text{red}} = \frac{(M+N) \cdot K \cdot e}{N_{\text{banks}} \cdot W} \cdot \tau \tag{3}$$

The pipelined latency is:

$$T_{\text{latency}} = t_{\text{fill}} + \max((n_{\text{rb}} - 1) \cdot t_{\text{fill}},\ T_{\text{pe}} + T_{\text{red}}) \tag{4}$$

D. Score@V on GPU

After Q@K produces float16 scores S, the output is $O = S \cdot V$. Since S is float16 and V must be dequantized from 2-bit, this operation is float16-intensive. GPU provides 989 TFLOPS (B100) — far exceeding per-bank PE throughput. V dequantization ($v = s_V(\hat{v} - z_V)$) and matrix multiply are efficiently handled by GPU tensor cores.

E. Heterogeneous Timeline

GPU and PIM advance independent timelines:

$$T_{\text{total}} = T_{\text{PIM}}^{\text{QK}} + T_{\text{GPU}}^{\text{softmax}} + T_{\text{GPU}}^{\text{SV}} \tag{5}$$

While Q@K, softmax, and score@V are sequential due to data dependencies, GPU projection operations (QKV projection, output projection) overlap with PIM attention execution.

F. KV Cache Quantization Overhead

In decode mode, each new token's K and V must be quantized before cache storage. We model this on GPU:

$$T_{\text{quant}} = \max\left(\frac{2 \cdot d_{\text{kv}} \cdot e}{\text{BW}_{\text{GPU}}},\ \frac{4 \cdot d_{\text{kv}}}{\text{FLOPS}_{\text{GPU}}} \right) \tag{6}$$

where $d_{\text{kv}} = n_{\text{kv\_heads}} \cdot d_{\text{head}}$. For Llama-3-8B ($d_{\text{kv}} = 1024$), this overhead is $< 0.01\mu s$ per token — negligible.

IV. Experimental Evaluation

A. Setup

Model and Hardware. Llama-3-8B (32 layers, $d_{\text{model}} = 4096$, $n_{\text{heads}} = 32$, $n_{\text{kv\_heads}} = 8$, $d_{\text{head}} = 128$), context length 16,400, 128 decode steps, batch size 64. GPU: B100 (989 TFLOPS FP16, 3.35 TB/s HBM). PIM: 2048 banks (8 stacks $\times$ 256), 2KB row buffer, PE 16B/cycle at 2GHz, 32 GB/s per-bank bandwidth. Asymmetric quantization: $z_Q = 1.0$, $z_K = 2.0$.

Simulator. Cycle-accurate C++ simulator extending LLM-Simulator [6] with configurable near-bank PIM units. Reports per-stage timing (Q@K, softmax, score@V) and per-bank component times (rowbuffer vs. PE). All PIM latencies verified against analytical cycle counts (ratio $1.00\times$).

Configurations. Six configurations evaluated:

TABLE I
PER-STEP ATTENTION LATENCY (μs).

Config	Q@K	S@V	Gen	pim_rb	pim_pe
FP16 GPU	97	—	95	—	—
FP16 PIM	66	66	132	131	66
2-bit GPU	48	—	48	—	—
2-bit Hyb.	33	34	67	33	17
2-bit All	33	33	66	66	33
2-bit Deq.	66	66	132	66	99

TABLE II
END-TO-END LATENCY (μs) FOR 128 STEPS × 32 LAYERS.

Config	Total	Per Layer
FP16 GPU	5,623	176
FP16 PIM	6,788	212
2-bit GPU	2,769	87
2-bit Hyb.	3,460	108
2-bit All	3,443	108
2-bit Deq.	5,555	174

1) **FP16 GPU**: All GPU, float16 KV.
2) **FP16 PIM**: All PIM, float16 KV (reference).
3) **2-bit GPU**: 2-bit KV, all GPU.
4) **2-bit Hybrid** (PIM-Attn): Q@K on PIM, softmax+score@V on GPU.
5) **2-bit All-PIM**: Both Q@K and score@V on PIM.
6) **2-bit Dequant**: Dequantize to FP16 in PIM before GEMV.

B. Per-Step Attention Latency

Table I presents per-step attention latency (16 seqs/DP group).

Key findings:

- PIM-Attn (2-bit Hybrid) achieves $2.1\mu s$/seq Q@K vs. $3.0\mu s$/seq for 2-bit GPU — $1.4\times$ speedup. Against FP16 GPU ($6.0\mu s$/seq), the advantage is $2.9\times$.
- Dequantize-then-compute is inefficient: $3\times$ PE increase (99 vs. 33 μs), doubling Q@K latency. This validates direct 2-bit compute.
- Row-buffer time dominates: pim_rb accounts for 50–66% of latency, indicating memory-bandwidth-bound execution at 2048 banks.
- Score@V benefits from GPU: at $34\mu s$ (GPU) vs. $33\mu s$ (PIM), GPU float16 throughput offsets data movement overhead.

C. PIM Cycle Verification

For FP16 PIM at $N_{\text{banks}} = 2048$: per-GEMV data = 4.26MB, per-bank = 2082B, requiring 2 row-buffer fills. Row-buffer time: $2 \times 64\text{ns} = 128\text{ns}$ per GEMV. PE time: $2082\text{B}/16 \times 0.5\text{ns} = 65.1\text{ns}$ per GEMV. Per-step (Q@K + score@V, $\times 4$ group, $\times 8$ heads, $\times 16$ seqs): RB $131.1\mu s$, PE $66.6\mu s$. **Reported:** $131.1\mu s$ **and** $66.5\mu s$ **(ratio 1.00×), confirming simulator fidelity.**

D. Asymmetric Quantization Overhead

With asymmetric quantization, reduction terms add PE time proportional to $(M+N)\cdot K$. For decode mode ($M=1, N=16528$), this matches the main GEMV data volume, resulting in $\sim 1.8\times$ PE time increase. In prefill mode with larger M, the relative overhead diminishes.

E. End-to-End Throughput

Table II reports total latency for 128 steps × 32 layers.

2-bit GPU achieves the lowest latency ($2{,}769\mu s$) due to reduced memory traffic combined with GPU throughput. PIM-Attn ($3{,}460\mu s$) is $1.25\times$ slower than 2-bit GPU but offers the advantage of keeping KV cache in PIM memory, enabling larger batch sizes and longer contexts exceeding GPU memory capacity.

V. RELATED WORK

KV Cache Quantization. KiVI [3] is the first tuning-free asymmetric 2-bit quantization for KV cache. FlexGen [7] and H2O [8] apply 4-bit round-to-nearest quantization. Atom [9] combines low-bit quantization with token-wise pruning. KVQuant [10] targets million-token contexts. Our work builds on KiVI's asymmetric formulation but addresses the *computational* challenge.

Processing-in-Memory for ML. Ambit [5] demonstrates bulk bitwise DRAM operations. Newton [11] proposes near-bank PIM for DNN training. Duplex [6] combines logic-die PIM with GPU for heterogeneous attention, but focuses on float16 operations without quantization support. PIM-Attn specifically targets quantized attention with 2-bit near-bank compute.

Attention Optimization. FlashAttention [12] reduces HBM accesses via tiling. Multi-query attention [13] and grouped-query attention [14] reduce KV heads. Our approach is orthogonal: we optimize the hardware mapping for quantized execution.

Mixed-Precision Inference. LLM.int8() [15] and SmoothQuant [16] use mixed-precision for weights and activations. PIM-Attn introduces spatial heterogeneity across PIM and GPU for attention stages.

To our knowledge, PIM-Attn is the first to: (1) decompose asymmetric quantized attention for PIM-GPU execution, (2) implement 2-bit near-bank compute for the decomposed operations, and (3) provide cycle-accurate analytical verification.

VI. CONCLUSION

We presented PIM-Attn, a heterogeneous attention framework that maps decomposed asymmetric quantized attention operations to near-bank PIM. Our key insight is that the asymmetric dot product $Q \cdot K \approx s_Q s_K[\sum \hat{q}\hat{k} - z_K \sum \hat{q} - z_Q \sum \hat{k} + dz_Q z_K]$ decomposes into 2-bit-compatible operations ideal for near-bank PIM, while float16-intensive score@V operations remain on GPU. Cycle-accurate simulation on Llama-3-8B demonstrates $1.4\times$ per-sequence Q@K speedup over GPU.

Future work includes extending to prefill-mode attention and integrating with tiling-based memory optimization.

REFERENCES

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[2] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," *Proceedings of Machine Learning and Systems*, vol. 5, 2023.

[3] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *International Conference on Machine Learning (ICML)*, 2024.

[4] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Processing data where it makes sense: Enabling in-memory computation," in *MicPro*, 2020.

[5] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-memory accelerator for bulk bitwise operations using commodity DRAM technology," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017.

[6] S. Yun, K. Kyung, and J. Cho, "Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.

[7] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *International Conference on Machine Learning (ICML)*, 2023.

[8] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H_2O: Heavy-hitter oracle for efficient generative inference of large language models," 2023.

[9] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate LLM serving," *Proceedings of Machine Learning and Systems*, vol. 6, 2024.

[10] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," *arXiv preprint arXiv:2401.18079*, 2024.

[11] L. Ke, U. Gupta, B. Y. Cho, D. Brooks, V. Chandra, U. Diril, A. Firoozshahian, K. Hazelwood, B. Jia, H.-H. S. Lee *et al.*, "Newton: A DRAM-maker's accelerator-in-memory for DNN training," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.

[12] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[14] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[15] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*, 2023.